

National Technical University of Athens
School of Electrical and Computer Engineering
Department of Computer Science

Computing Systems Laboratory
<http://www.cslab.ece.ntua.gr>

Operating Systems

6th Semester, Academic Year 2024-2025

1st Laboratory Exercise

System Calls, Processes, and Interprocess Communication

Computing Systems Laboratory, NTUA
February 2025

1 Reading and Writing Files in C Using System Calls (30%)

In the directory `/home/oslab/code/char-count`, you will find C code (and a Makefile for the entire exercise) that counts the occurrences of a character in an input file and writes the result to an output file. The filenames and the character to search for are provided by the user via the command line.

Note: Observe that when calling functions with uncertain outcomes (e.g., `fopen`), we use error-handling code. Follow this practice in all subsequent exercises.

Task: Implement a program with identical functionality, but using system calls instead of C library functions. Hints: `man 2 open`, `man 2 read`, `man 2 write`, `man 2 close`, `man 2 exit`.

Optional: The provided code may fail if the user does not supply the correct command-line parameters. Extend your program to perform necessary checks and provide usage instructions.

2 Process Creation (30%)

Task: Extend the program from Question 1 as follows:

1. Create a child process that greets the world, printing its own process ID (PID) and its parent's PID. The parent should print the child's PID and wait for its termination. Hints: `man fork`, `man wait`, `man getpid`, `man getppid`.
 2. Define a variable, e.g., `x`, in the parent before creating the child. Assign different values to `x` in the parent and child, then print its value. What do you observe?
 3. Assign the task of searching for the character in the file to the child process (but not the rest of the file handling).
 4. Create a child process that executes the code provided in Question 1. Hint: `man execv`.
-

3 Interprocess Communication (30%)

Task: Extend the program from Question 2 to create `P` child processes (`P` can be defined as a constant in your program). These processes will search for the character in the file in parallel, and the parent process will collect and print the total result. When the program receives a Control+C (SIGINT signal) from the keyboard, instead of terminating, it should print the total number of processes currently searching the file.

4 Parallel Character Counting Application (10%)

Task: Implement a complete parallel character-counting application for a file. The application will accept the filename and the search character from the user. During execution, it should accept user commands for:

- a) Adding/removing worker processes (searchers).
- b) Displaying information about the workers participating in the search.
- c) Updating the search progress (completion percentage and number of characters found so far).

The application should be modular and consist of three independent components:

- **Front-end:** A process that communicates with the user, accepts commands, and handles them either directly or by forwarding them to the dispatcher.
- **Dispatcher:** A process responsible for distributing work to workers. It receives commands from the front-end to add/remove workers, divides the work among available workers in a round-robin fashion, collects partial results, and returns the total result to the front-end.

- **Worker:** The program should dynamically adjust the number of workers. Each worker repeatedly receives a portion of the file to search (starting byte position and number of bytes to search) and returns the result to the dispatcher.

Additional Features:

- The dispatcher should manage a work pool and worker list to handle cases where a worker terminates unexpectedly (e.g., due to a signal).
- Communication between components (front-end, dispatcher, workers) should use pipes.
- Intermediate results should be displayed using a combination of signals and pipes.

Implementation Details:

- Use `sprintf` and byte streams for pipe communication, with format specifiers like `%d`, `%5d`, etc.
- Non-blocking `read` system calls may be necessary.
- Investigate what happens if a system call (e.g., `read` or `write`) is interrupted or if the other end of a pipe belongs to a terminated process.
- Study examples of `execv` with parameter passing to child processes.
- Artificial delays (e.g., `sleep`, `usleep`) may be added to the dispatcher or workers for real-time execution monitoring.